Guía de Estudio: TP1 de Procesamiento de Imágenes I

Ecualización local de histograma y validación de planillas de calificaciones. Procesamiento de Imágenes I (IA 4.4), TUIA, FCEIA, UNR. Año 2026, 2° semestre. Grupo 8.

Nivel asumido: universitario, tecnicatura en inteligencia artificial. La guía sale del enunciado, del informe, de los dos scripts del grupo (`problema1.py` y `problema2.py`) y de los cinco documentos `Problema*_explicacion.md` del repositorio local, que además citan los archivos de la cátedra usados en cada paso (sección 6). Las definiciones generales son las habituales de la disciplina; conviene contrastarlas con la notación de las clases.

Objetivo de Aprendizaje

Al terminar vas a poder explicar con tus palabras cómo funcionan la ecualización local de histograma y la validación automática de planillas por procesamiento de imágenes, justificar cada decisión del TP y reconstruir los dos scripts paso a paso.

Conceptos Clave (El "Qué" y el "Por qué")

- **Histograma**: cuenta cuántos píxeles hay de cada nivel de gris (de 0 a 255 en 8 bits). Muestra cómo se reparte el brillo de la imagen.
- **Ecualización de histograma**: transforma los niveles de gris para que ocupen todo el rango disponible y suba el contraste. Cada nivel pasa a valer proporcionalmente a la fracción de píxeles que tienen ese nivel o uno menor (histograma acumulado). En el TP se usa `cv2.equalizeHist`.
- **Histograma normalizado y acumulado (CDF)**: dividir el histograma por el total de píxeles da la probabilidad de cada nivel, y sumar esas probabilidades de a una da la curva acumulada. La ecualización usa esa curva como función de transformación.
- **Ecualización global**: usa el histograma de toda la imagen. Si un detalle ocupa pocos píxeles y su gris casi coincide con el del fondo, el histograma apenas lo registra y el detalle sigue invisible.
- **Ecualización local**: calcula el histograma solo dentro de una ventana que se desplaza píxel a píxel y usa esa transformación únicamente para el píxel central de la ventana. Así cada zona se ecualiza con sus propios vecinos.
- **Ventana MxN**: M es el alto y N el ancho. En el script ambos son enteros positivos e impares, para que exista un píxel central.
- **Borde replicado** (`cv2.BORDER_REPLICATE`): copia hacia afuera los valores del borde de la imagen, para que la ventana no se salga cuando el píxel central está en un extremo. No agrega niveles de gris que la imagen no tiene, a diferencia de rellenar con ceros.
- **Umbralado**: convierte la imagen en blanco y negro con una condición, por ejemplo `img < 150`. El texto y las líneas quedan en verdadero.
- **Proyección por filas y columnas**: suma de píxeles oscuros de cada fila o columna. Las líneas de una tabla producen picos muy altos.
- **Componente conectada**: grupo de píxeles vecinos que forman una región. En las planillas cada carácter es una componente. Con conectividad 8 los vecinos en diagonal también cuentan.
- **Contorno con madre**: al buscar contornos con `cv2.RETR_TREE` se obtiene una jerarquía. Un contorno que tiene madre está dentro de otro, y eso indica un agujero (la R tiene uno, la L no).
- **Validación de formato**: se comprueba que cada campo cumpla su regla de cantidad de caracteres y palabras, sin leer qué dice.

Desarrollo Estructurado

1. Visión general del TP

Problema	Qué se pide	Idea central	Script
1	a) función de ecualización local, b) detalles ocultos de una imagen, c) efecto del tamaño de ventana	Histograma calculado dentro de una ventana que recorre la imagen	`problema1.py`
2	a) validar cada campo de una planilla, b) imagen con alumnos que no aprobaron, c) CSV, d) repetir con 4 planillas	Encontrar la tabla, aislar cada celda, contar caracteres y palabras	`problema2.py`

Se usan tres librerías: numpy (arreglos y sumas), opencv-python (bordes, ecualización, componentes conectadas y contornos) y matplotlib (figuras del Problema 1).

2. Problema 1: ecualización local de histograma

2.1 Marco teórico

El histograma de una imagen en grises cuenta cuántos píxeles hay de cada nivel de intensidad. Con L = 256 niveles (8 bits), `h(r_k) = n_k` es la cantidad de píxeles de nivel r_k, con k de 0 a L-1. Si se divide por el total de píxeles se obtiene el histograma normalizado, `p(r_k) = n_k / total`, que se lee como la probabilidad de cada nivel. En la cátedra aparecen tres formas de calcularlo:

Función	Cuándo conviene
`cv2.calcHist()`	La más rápida; admite varios canales y máscaras
`np.histogram()`	Velocidad media; sirve para cualquier tipo de dato, pero sin máscaras ni varios canales
`plt.hist()`	La más lenta; está pensada solo para visualizar

La ecualización global transforma cada nivel con la función de distribución acumulada (CDF) escalada al rango de salida:

```
s_k = T(r_k) = (L - 1) * suma de p(r_j), para j de 0 a k
```

En la cátedra se escribe como `histn = hist / img.size` y `cdf = histn.cumsum()`, y `cv2.equalizeHist(img)` lo hace todo junto. Como usa todos los píxeles de la imagen, un detalle chico cuyo gris casi coincide con el de su entorno pesa muy poco en el histograma y no se resalta.

La ecualización local aplica la misma idea dentro de una ventana MxN centrada en cada píxel: se calcula el histograma solo con los píxeles de la ventana y la transformación resultante se aplica únicamente al píxel central. Como la curva sale de los vecinos, diferencias mínimas dentro de una zona casi uniforme se estiran a todo el rango de 0 a 255, y por eso aparecen los detalles escondidos.

Para que la ventana tenga un único píxel central tiene que haber la misma cantidad de píxeles a cada lado: M = 2m + 1 y N = 2n + 1. Con un tamaño par el centro queda entre dos píxeles, y por eso M y N son impares.

Cuando el píxel central está cerca del borde, parte de la ventana cae fuera de la imagen. OpenCV ofrece varios rellenos, que se ven en una fila de ejemplo abcdefgh:

Tipo de borde	Resultado
`BORDER_REPLICATE`	`aaaaaa|abcdefgh|hhhhhhh`
`BORDER_REFLECT`	`fedcba|abcdefgh|hgfedcb`
`BORDER_REFLECT_101`	`gfedcb|abcdefgh|gfedcba`

Tipo de borde	Resultado
`BORDER_WRAP`	`cdefgh|abcdefgh|abcdefg`
`BORDER_CONSTANT`	`iiiiii|abcdefgh|iiiiiii` (con algún valor i)

Se usa `BORDER_REPLICATE` porque copia el valor del borde y no agrega niveles que la imagen no tiene. Con `BORDER_CONSTANT` en cero se meterían píxeles negros artificiales en el histograma de las ventanas del borde y se alteraría la ecualización.

2.2 El punto de partida

La imagen (`Imagen_con_detalles_escondidos.tif`) mide 256x256 píxeles y tiene 8 bits, pero solo usa 15 niveles de gris, en dos grupos. Hay cinco cuadrados oscuros de 62x62 píxeles y, dentro de cada uno, una figura cuyo gris se diferencia entre 5 y 10 niveles del fondo del cuadrado. Por eso a simple vista no se ve nada.

Región	Niveles	Píxeles
Fondo claro	226, 227 y 228	1175, 43946 y 1195
5 cuadrados oscuros de 62x62	0 a 11	5 x 3844 = 19220

El fondo claro no es del todo uniforme: tiene píxeles sueltos de 226 y 228 sobre un fondo de 227. Dentro de cada cuadrado el fondo es 0 (con algunos píxeles sueltos de nivel 1) y el detalle está dibujado con niveles entre 1 y 11. Los 19220 píxeles oscuros son el 29,3 % de la imagen: con ecualización global quedan todos juntos al principio del histograma acumulado y reciben valores bajos, así que los detalles siguen invisibles.

2.3 Punto a: la función `ecualizacion_local(img, M, N)`

1. Calcular `m = M // 2` y `n = N // 2`.
2. Agregar el borde con `cv2.copyMakeBorder(img, m, m, n, n, cv2.BORDER_REPLICATE)`. Con una ventana de 21x21 se agregan 10 píxeles por lado y la imagen pasa de 256x256 a 276x276.
3. Para cada píxel (i, j), tomar la ventana MxN centrada en él.
4. Ecualizar la ventana con `cv2.equalizeHist`.
5. Guardar en la imagen de salida solo el píxel central de la ventana ya ecualizada.

Por el borde agregado, el píxel (i, j) de la imagen original queda en (i + m, j + n) de la imagen con borde. Por eso `img_borde[i:i + M, j:j + N]` es exactamente la ventana centrada en (i, j), y su centro está en `[m, n]`. M es el alto (filas agregadas arriba y abajo) y N el ancho (columnas agregadas a los costados), así que también funcionan ventanas rectangulares.

El script valida que M y N sean enteros positivos impares y avisa con un error si no lo son. Acepta enteros de NumPy (`np.int64`) además de int, útil si los tamaños se generan con `np.arange`. Como la ecualización se hace una vez por píxel, en una imagen de 256x256 son 65536 llamadas, y el costo crece con el tamaño de la ventana.

Ejemplo numérico: para la ventana de 21x21 centrada en (i = 212, j = 207), los niveles van de 0 a 11 y el píxel central vale 10. Después de ecualizar la ventana, ese píxel pasa a valer 247 y es el que va a la imagen de salida.

Prueba	Resultado
Imagen del TP, ventana 15x15	Corre sin errores en unos 0,14 s; salida uint8 del mismo tamaño
Ventanas 3x3, 3x7 y 1x1	Salida del mismo tamaño que la entrada
M y N como `np.int64`	Aceptados

Prueba	Resultado
M par (4x3), M = 0 y M = 3.0 (float)	Lanzan `ValueError`
Imagen sintética con un cuadrado oculto, 5 niveles más claro que el fondo	Tras la ecualización local la diferencia pasa a 255

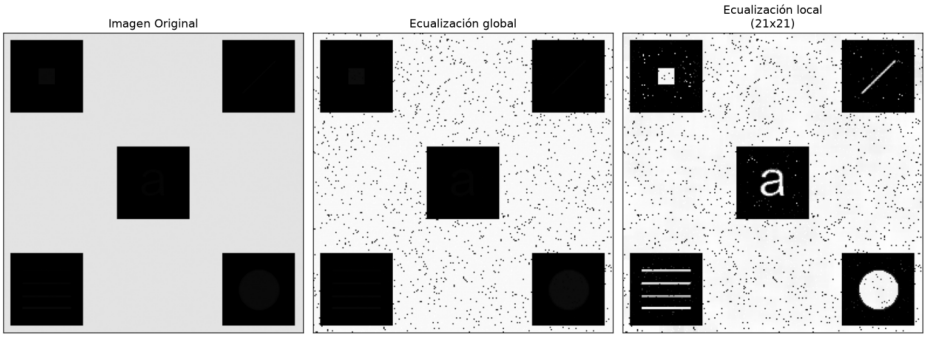


Figura 1. Imagen original, ecualización global y ecualización local con ventana de 21x21. La global deja los cuadrados casi igual; la local muestra una figura clara dentro de cada uno.

2.4 Punto b: detalles ocultos

1. **Ubicar las zonas.** Las coordenadas de los cinco cuadrados (lado `L = 62`) se midieron a mano y están en la tabla `ZONAS` del script, junto con el detalle de cada uno. El enunciado pide analizar una sola imagen, así que no hace falta detectarlas automáticamente.
2. **Ecualizar localmente** toda la imagen con la ventana elegida en el punto c: 21x21.
3. **Mostrar** la imagen completa (original, global y local) y un zoom de cada zona antes y después.
4. **Informar** por consola el detalle de cada zona. La descripción sale de mirar la figura: el script la muestra como título de cada recorte y la imprime.

Zona	Posición (x, y)	Detalle oculto
Superior izquierda	(6, 6)	Un cuadrado pequeño
Superior derecha	(187, 6)	Una línea diagonal (de abajo a la izquierda hacia arriba a la derecha)
Centro	(97, 97)	La letra "a" minúscula
Inferior izquierda	(6, 188)	Cuatro líneas horizontales paralelas
Inferior derecha	(187, 188)	Un círculo

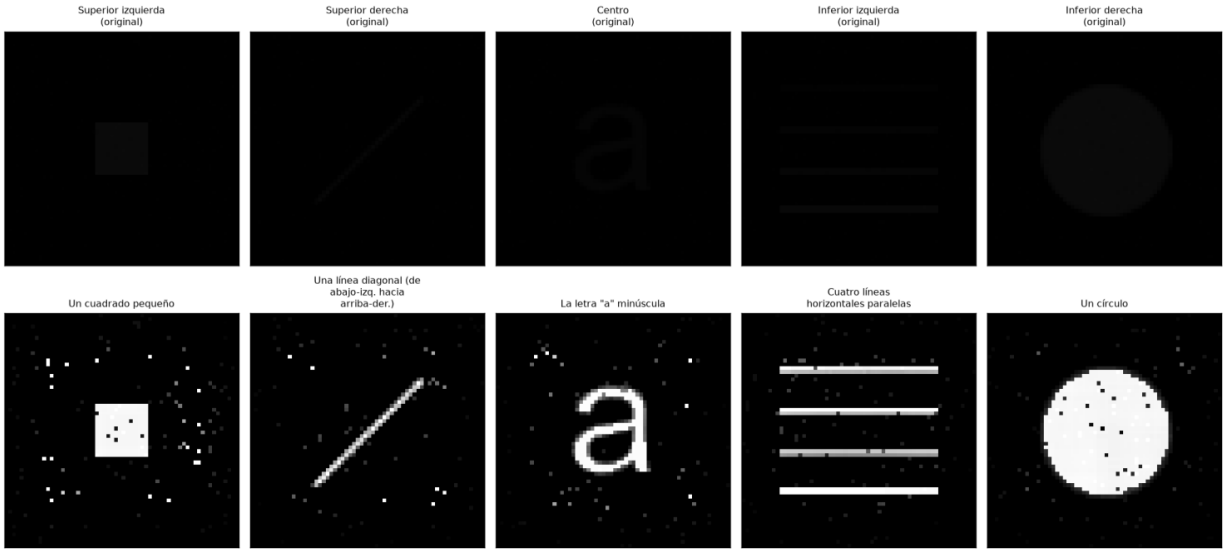


Figura 2. Cada cuadrado antes (arriba) y después (abajo) de la ecualización local de 21x21.

2.5 Punto c: influencia del tamaño de la ventana

El script prueba ventanas cuadradas de 3, 7, 15, 21, 31, 61 y 101 de lado, y rectangulares de 3x31, 31x3, 1x61 y 61x1. Las figuras comparten los ejes, así que al hacer zoom se ve la misma región con todas las ventanas. La comparación es visual: se miran las figuras y se describe lo que aparece.

Ventana	Qué se ve
3x3	El fondo claro se llena de pequeños anillos oscuros. El cuadrado pequeño y el círculo aparecen solo con su contorno.
7x7	Manchas negras grandes en el fondo claro. El cuadrado se ve relleno y el círculo tiene zonas negras en su interior.
15x15 y 21x21	Los cinco detalles se ven completos y el fondo queda casi limpio. Con 15x15 hay algunas marcas negras cortas más.
31x31	Los detalles siguen visibles. Las cuatro líneas horizontales se ven más grises.
61x61	Los detalles pierden brillo. En varias figuras el centro queda más claro que los bordes.
101x101	Los detalles casi no se distinguen, igual que con la ecualización global.
Rectangulares	Aparecen rayas en el fondo con la orientación de la ventana: horizontales con 3x31 y 1x61, verticales con 31x3 y 61x1.

Por qué pasa, según el tamaño:

- **Ventanas muy chicas (3x3 y 7x7).** Con 3x3 los detalles grandes aparecen huecos. Cuando la ventana cae entera dentro del detalle, todos sus píxeles tienen casi el mismo nivel y no hay nada que estirar; solo en el borde, donde la ventana mezcla detalle y fondo, aparece contraste. Los detalles finos (las líneas de 2 píxeles, la "a") se ven bien porque la ventana siempre toca un borde. En el fondo claro hay ventanas con solo 227 y 228, o solo 226 y 227: `cv2.equalizeHist` lleva el nivel más bajo presente a 0, así que una diferencia de un nivel pasa a ser negro contra blanco.
- **Ventanas intermedias (15x15 a 31x31).** Los cinco detalles se ven completos. En el fondo quedan puntos negros sueltos: casi toda ventana incluye algún píxel de 226, que es siempre el nivel más bajo del fondo. Son los 1175 píxeles de 226 de la imagen original (el 2,5 % del fondo), ruido de la imagen que aparece oscuro incluso con la ecualización global. Se eligió 21x21 porque el fondo ya no tiene manchas, los cinco detalles se ven bien y dentro de los cuadrados quedan menos puntos sueltos que con 15x15.

- **Ventanas grandes (61x61 en adelante).** Cuando la ventana se sale del cuadrado de 62x62 entran píxeles del fondo claro. En el histograma acumulado de la ventana esos píxeles ocupan la parte de arriba, y todos los niveles oscuros (0 a 11) quedan comprimidos en el rango de 0 a 255 por p, donde p es la fracción de píxeles oscuros de la ventana. Cuanto más grande es la ventana, menor es p y más oscuro queda el detalle. Con 61x61 aparece además un efecto de viñeta: la ventana centrada en el borde del detalle incluye más fondo claro, y el centro queda más claro que los bordes.
- **Regla práctica.** Cada zona empieza a apagarse cuando la mitad de la ventana (M // 2) supera la distancia del detalle al borde de su cuadrado. Las cuatro líneas de abajo a la izquierda están a unos 10 píxeles del borde y con 31x31 ya se ven más apagadas. El cuadrado pequeño está a unos 24 píxeles, se sigue viendo bien con 31x31 y recién se apaga con 61x61. Son medidas del grupo.
- **Ventanas rectangulares.** Con 3x31 (ventana horizontal) las líneas horizontales se mantienen nítidas; con 31x3 (vertical) se degradan, porque la ventana sale del cuadrado por arriba y por abajo y toma fondo claro. Con 1x61 y 61x1 la ventana sale del cuadrado en una dirección y apaga los detalles, sobre todo la línea diagonal. Si el detalle tiene una dirección conocida conviene una ventana alargada en esa dirección; si no se sabe, una ventana cuadrada.
- **Costo.** El tiempo crece con el tamaño de la ventana, porque en cada píxel se calcula el histograma de M x N valores.

Conclusión: el tamaño tiene que quedar entre dos límites que dependen de la imagen. El mínimo es una ventana lo bastante grande para tomar a la vez parte del detalle y parte de su fondo; si no, solo se ven bordes y el ruido se amplifica. El máximo es una ventana lo bastante chica para no salirse de la zona del detalle; si no, el fondo claro domina el histograma y el resultado se parece al de la global. En esta imagen el rango útil va de 15x15 a 31x31, y 21x21 da el mejor equilibrio.

3. Problema 2: validación de planillas de calificaciones

3.1 Qué hay que verificar

Cada planilla es una imagen con una tabla de 20 registros y seis campos: Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3 y Condición Final. Hay cuatro planillas completas (`grade_sheet_1.png` a `grade_sheet_4.png`) de tamaños distintos. Los caracteres permitidos son las letras de la A a la Z (incluida la Ñ), los dígitos, el guion y la barra. El enunciado pide verificar solo el formato de cada campo; si el dato es real o válido queda fuera del TP. Alcanza con contar caracteres y palabras, que es lo que se puede sacar de la imagen con lo visto en la materia. En el código, c es la cantidad de caracteres y p la de palabras.

Campo	Regla del enunciado	Regla en el código
Legajo	8 caracteres en total, una sola palabra	`c == 8` y `p == 1`
Nombre y Apellido	Al menos dos palabras y no más de 12 caracteres	`p >= 2` y `c <= 12`
Parcial 1, 2 y 3	1 o 2 caracteres consecutivos, sin espacios	`1 <= c <= 2` y `p == 1`
Condición Final	Un único carácter	`c == 1`

3.2 Punto a: el recorrido de una planilla

1. **Umbralar.** Se lee la imagen en grises y se calcula `img < 150`, una matriz booleana donde el texto y las líneas quedan en verdadero. El texto y las líneas son casi negros y los bordes grises de la hoja valen entre 178 y 201, así que con 150 no se cuelan como si fueran líneas de la tabla. El 150 se eligió mirando los valores de la imagen.
2. **Sumar por filas y por columnas.** Con `np.sum` se obtiene cuántos píxeles oscuros tiene cada fila y cada columna. Las líneas de la tabla se destacan porque tienen muchísimos más.

3. **Fijar umbrales relativos.** Una fila es línea si supera 0,8 veces el máximo, y una columna si supera 0,6 veces el máximo. Son relativos porque las cuatro planillas miden distinto (1107x867, 1029x869, 1106x869 y 917x1082). La línea que está debajo de "Notas" es más corta que las demás y queda afuera, que es lo que se busca. El umbral de columnas es más bajo porque las divisiones entre Parcial 1, 2 y 3 son más cortas. Medido en las cuatro planillas, las líneas verticales llegan como mínimo al 83 % del máximo y ninguna otra columna pasa del 37 %, así que el 60 % toma las 8 con margen de los dos lados.
4. **Pasar de picos a posiciones.** Las líneas pueden tener más de un píxel de ancho, así que hay que encontrar su principio y su fin. La función `pulsos` usa `np.diff` para marcar los cambios y `np.argwhere` para obtener sus índices; a los inicios se les suma 1 y se agrupan de a pares con `reshape(-1, 2)`. El resultado es el mismo en las cuatro planillas: 22 líneas horizontales (el borde de arriba de la tabla, la línea debajo del encabezado y una debajo de cada uno de los 20 registros) y 8 verticales, que definen 7 columnas.
5. **Recortar las celdas.** Cada celda es el rectángulo entre dos líneas horizontales y dos verticales consecutivas, sin incluir las líneas: desde el fin de una línea más 1 hasta el inicio de la siguiente. Cada registro queda entre dos líneas seguidas a partir de la segunda. La columna Nro. no se valida.
6. **Contar caracteres.** En cada celda se buscan componentes conectadas con `cv2.connectedComponentsWithStats` (conectividad 8). La primera es el fondo y se saca. Se descartan además las de área 1 o menos, y cada componente que queda es un carácter. El filtro de área es tan permisivo porque hay caracteres diminutos: el guion del legajo tiene 4 o 5 píxeles de área y el punto de "1.0" tiene 3. Un umbral más alto se comía caracteres reales.
7. **Contar palabras.** Se ordenan las componentes de izquierda a derecha con `sorted` y se mide el hueco horizontal entre el final de una y el comienzo de la siguiente. Entre letras de una misma palabra el hueco medido va de 0 a 4 píxeles; entre palabras, de 10 a 13. Con un umbral de 6, si el hueco lo supera hay un espacio. Palabras = espacios + 1. Es la misma idea que separa letras buscando columnas vacías, un nivel más arriba: los huecos cortos separan letras y los largos separan palabras.
8. **Aplicar la regla del campo** con esos dos números e imprimir OK o MAL.

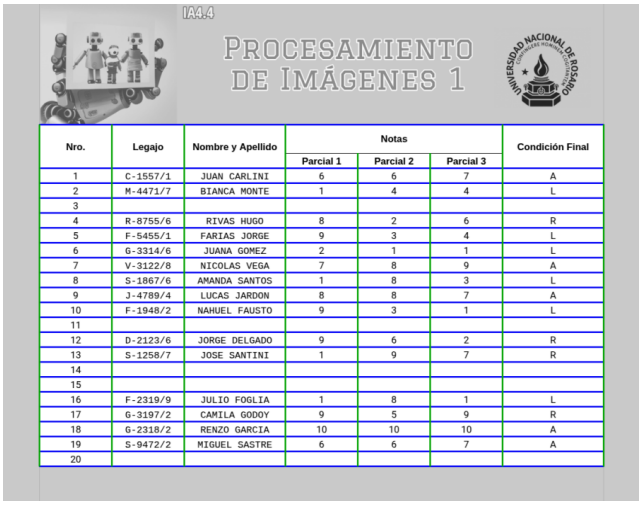
Una celda vacía cuenta 1 palabra y 0 caracteres, así que incumple todas las reglas y queda en MAL. La salida por pantalla sigue el formato del enunciado:

```
> Registro 1:
> Legajo: OK
> Nombre y apellido: MAL
> Parcial 1: OK
> Parcial 2: MAL
> ...
>
> Registro 2:
```

Se comparó a mano la salida de las cuatro planillas con lo que se ve en cada imagen (80 registros) y coincide. Algunos casos que confirman las reglas:

Planilla	Registro	Contenido	Resultado
1	1	Ejemplo del enunciado (C-1557/1, JUAN CARLINI, 6, 6, 7, A)	Todo OK
2	1	JULIAN CARLINI (13 letras)	Nombre MAL
2	4	R-87455/6 (9 caracteres)	Legajo MAL
2	19	S-94722/ 1 (tiene un espacio)	Legajo MAL; Parcial 3 = 100, MAL
3	8	AMANDASANTOS (una sola palabra)	Nombre MAL

Planilla	Registro	Contenido	Resultado
3	15	Parcial 2 = 1.0 (el punto cuenta como carácter)	Parcial 2 MAL
2	17	Condición "Rec"	Condición MAL
1	3, 11, 14, 15 y 20	Filas vacías	Todo MAL



Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1	C-1557/1	JUAN CARLINI	6	6	7	A
2	M-4471/7	BIANCA MONTE	1	4	4	L
3						
4	R-8755/6	RIVAS HUGO	8	2	6	R
5	F-5455/1	FARIAS JORGE	9	3	4	L
6	G-3314/6	JUANA GOMEZ	2	1	1	L
7	V-3122/8	NICOLAS VEGA	7	8	9	A
8	S-1867/6	AMANDA SANTOS	1	8	3	L
9	J-4789/4	LUCAS JARDON	8	8	7	A
10	F-1948/2	NAHUEL FAUSTO	9	3	1	L
11						
12	D-2123/6	JORGE DELGADO	9	6	2	R
13	S-1258/7	JOSE SANTINI	1	9	7	R
14						
15						
16	F-2319/9	JULIO FOGLIA	1	8	1	L
17	G-3197/2	CAMILA GODOY	9	5	9	R
18	G-2318/2	RENZO GARCIA	10	10	10	A
19	S-9472/2	MIGUEL SASTRE	6	6	7	A
20						

Figura 3. Planilla 1 con las líneas detectadas: 22 horizontales en azul y 8 verticales en verde. Las dos divisiones internas de Notas arrancan más abajo, por eso se usa un umbral de columnas más bajo.

3.3 Punto b: alumnos que no aprobaron

Hay que armar una única imagen con los alumnos de Condición Final L (libre) o R (recupera), mirando solo los registros que tienen los seis campos en OK. Cada alumno aporta el recorte del campo Nombre y Apellido con un indicador que diferencie a los R de los L. La dificultad es distinguir L, R y A sin reconocer texto: las tres miden lo mismo de alto (12 píxeles en las planillas 1, 2 y 3, y 10 en la 4), así que la altura no sirve. Lo que las diferencia es la forma, y para medirla se usan dos propiedades.

- **Columna izquierda llena.** La L y la R empiezan con un trazo vertical: la primera columna de su caja (bounding box, que sale de `stats`) está llena de arriba a abajo, y se mide sumando por columnas la letra recortada. La A arranca en diagonal y casi no tiene píxeles ahí.
- **Agujero.** La R tiene uno (la panza cerrada) y la L no. Se detecta con `cv2.findContours` y `cv2.RETR_TREE`, que devuelve la jerarquía [Next, Previous, First_Child, Parent]. El contorno de un agujero queda adentro del exterior de la letra, así que tiene madre (`Parent != -1`).

Las dos reglas juntas: columna izquierda incompleta, es A (u otro carácter) y se descarta; columna llena con agujero, R; columna llena sin agujero, L. Antes de escribir la regla se midieron las 56 celdas de Condición Final con un solo carácter en las cuatro planillas. En todas las L y R la primera columna mide exactamente el alto de la letra (12 píxeles, o 10 en la planilla 4) y en todas las A mide entre 1 y 3. Los agujeros dan 0 en todas las L y 1 en todas las A y R. No hubo ningún caso dudoso.

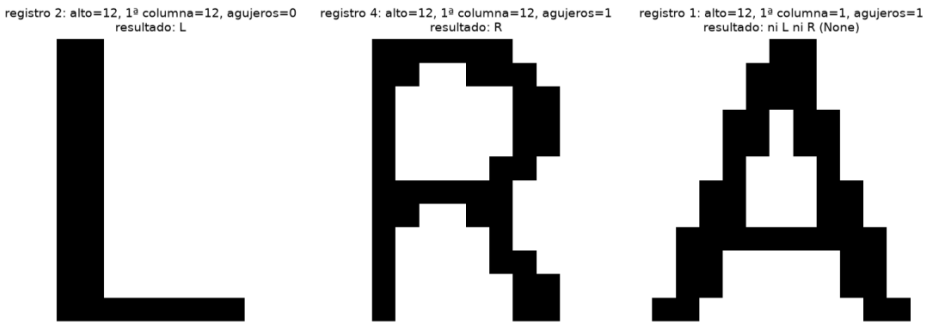


Figura 4. Letras L, R y A de la planilla 1 (registros 2, 4 y 1) con las medidas que usa el script.

Armado de la imagen de salida. Cada alumno es una fila: al recorte de Nombre y Apellido se le agrega un margen blanco con `cv2.copyMakeBorder` (borde constante de valor 255; 5 píxeles arriba, abajo y a la derecha, y 40 a la izquierda) y en ese margen se escribe la letra con `cv2.putText`: roja para R y azul para L. Las filas se apilan con `np.vstack`. El recorte sale de la imagen en grises y se pasa a 3 canales con `cv2.cvtColor` para poder dibujar en color; los colores van en BGR, que es el orden de OpenCV. Si ningún alumno cumple, `np.vstack` de una lista vacía daría error, así que se crea con `np.full` una imagen chica con el texto "Sin alumnos L/R" y siempre hay una única salida por planilla. Se guarda como `no_aprobados_grade_sheet_<id>.png`.

Planilla	Con todo OK y L o R	Alumnos
1	10	L: Bianca Monte, Farias Jorge, Juana Gomez, Amanda Santos, Nahuel Fausto, Julio Foglia. R: Rivas Hugo, Jorge Delgado, Jose Santini, Camila Godoy
2	3	L: Bianca Monte, Farias Jorge, Amanda Santos
3	0	Imagen con "Sin alumnos L/R": ningún registro tiene los seis campos bien
4	4	L: Nicolas Vega, Miguel Sastre. R: Jorge Delsio, Camila Godoy

En la planilla 4 también hay registros con todo OK y condición A (Juana Gomez, Renzo Garcia) y quedan afuera, como corresponde. Los registros con algún campo MAL no aparecen aunque su condición sea L o R: por ejemplo, Nahuel Fausto en la planilla 2 tiene "LL".

3.4 Puntos c y d: CSV y ciclo sobre las cuatro planillas

El CSV `validacion_grade_sheet_<id>.csv` no requiere procesar nada nuevo de la imagen: `validar_planilla` ya devuelve la lista de OK y MAL de cada registro en el orden de la planilla, y escribir el archivo es volcar esa lista. Tiene una primera línea con los nombres de las columnas, copiados del enunciado, y una línea por registro con su número (1 a 20) como ID y los seis resultados separados por comas (Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3 y Condición Final). Las filas vacías de la planilla también van, con todo MAL, porque el enunciado pide una fila por registro y el ID tiene que coincidir con el orden. Se escribe con `open` y `write`, sin librerías extra.

El bloque principal recorre las planillas con un `for`. Para cada una llama una sola vez a `validar_planilla` (punto a), le pasa los datos a `imagen_no_aprobados` (punto b), guarda el CSV (punto c) e imprime un resumen: cuántos registros tienen los seis campos bien, cuántos MAL hay en cada campo y cuántos alumnos quedaron en R y en L. Los nombres se arman con el índice (`grade_sheet_{k}.png` para k de 1 a 4), con lo que queda afuera `grade_sheet_empty.png`, que no tiene registros. Resultados:

Planilla	Registros correctos	MAL por campo (Leg / Nom / P1 / P2 / P3 / Cond)	No aprobados
grade_sheet_1	15 de 20	5 / 5 / 5 / 5 / 5 / 5	4 R, 6 L
grade_sheet_2	3 de 20	8 / 9 / 7 / 4 / 7 / 8	0 R, 3 L
grade_sheet_3	0 de 20	10 / 9 / 7 / 5 / 8 / 8	Ninguno
grade_sheet_4	6 de 20	6 / 7 / 4 / 5 / 5 / 6	2 R, 2 L

- La planilla 1 es la limpia: los únicos MAL son las 5 filas vacías (registros 3, 11, 14, 15 y 20), que fallan en todos los campos.
- La planilla 3 no tiene ningún registro completo. Los errores son de los mismos tipos que en la 2: legajos con dígitos de más o de menos o con espacios, nombres de una sola palabra o demasiado largos, notas vacías o de tres caracteres ("100", "AUS") y condiciones como "LL", "RR" o "Rec".

4. Decisiones y limitaciones

- El espacio entre palabras no cuenta como carácter del nombre, porque no es una componente conectada. El enunciado no aclara si cuenta, así que queda como consulta pendiente para la cátedra. Si hubiera que contarlo, alcanza con sumar `n_pal - 1` a los caracteres, y cambiaría el resultado de nombres como AMANDA SANTOS (12 letras, que hoy da OK) o NAHUEL FAUSTO.
- La Ñ se dibuja con una N y una tilde separada, y el script cuenta esa tilde como una componente más. En estas planillas no aparece ninguna Ñ, así que no se resolvió; habría que unir las componentes que se superponen en x.
- Una fila vacía da todo MAL, porque ningún campo vacío cumple su regla. Es lo que sale de aplicar el enunciado tal cual.
- Los umbrales del Problema 2 (150, 80 % del máximo para filas, 60 % para columnas, área 1 y espacio de 6 píxeles) no salen de ningún archivo de la cátedra. La consigna dice "definir un umbral acorde" y se ajustaron mirando las cuatro planillas; con otra tipografía u otra escala podrían fallar.
- El criterio de la columna izquierda llena para separar la A de la L y la R es del grupo: las herramientas (suma por columnas y bounding box) están en el material, pero la idea de usarlas así no. Si apareciera otra letra con trazo vertical a la izquierda (B, D, P), se clasificaría como L o R según tenga agujero o no. Como el enunciado solo habla de L y R, no se cubre.
- El formato de la imagen de no aprobados (apilado vertical, letra roja o azul a la izquierda) y el lugar donde se guardan las imágenes y los CSV (al lado de cada planilla) son elecciones del grupo. El enunciado pide "algún indicador" y no dice dónde guardar.
- Las posiciones de los cuadrados del Problema 1 están escritas a mano en la tabla `ZONAS`, y el script no encuentra zonas en otra imagen. La ventana de 21x21 se eligió mirando las figuras; el script no mide contraste ni ruido por ventana.
- En el Problema 2 el script no lee caracteres: cuenta componentes, mide huecos y compara la forma de tres letras.
- La ecualización local amplifica el ruido, como se vio con los píxeles de nivel 226 del fondo.

5. Cómo se entrega

La entrega es el link a un repositorio git con la resolución completa. Tiene que incluir los scripts en formato `.py`, un `README.md` con las versiones de las librerías y las instrucciones para ejecutar cada script, y el informe en PDF con la descripción de los ejercicios, el análisis de los problemas, las técnicas, las conclusiones y capturas de los pasos intermedios. Solo se aceptan repositorios con archivos `.py`, `.pdf` y `.md`. Las versiones con las que se ejecutó y verificó todo son Python 3.13.16, numpy 2.5.3, opencv-python 5.0.0.93 y matplotlib 3.11.2.

6. De dónde sale cada técnica

Los documentos de explicación del repositorio indican qué archivo de la cátedra respalda cada paso. Sirve para saber qué podés defender con "esto lo vimos en clase" y qué fue una decisión del grupo.

Técnica	Archivo de la cátedra
Lectura en grises con `cv2.imread`, `cv2.equalizeHist`, relación entre histograma y CDF, subplots con `sharex/sharey` y `vmin/vmax`	`PI_U2_Transformacion_y_Filtrado_p1.py` y diapositivas de U2
Tipos de borde y elección de `BORDER_REPLICATE`	`PI_U2_Transformacion_y_Filtrado_p2.py` y la AYUDA del enunciado (`cv2.copyMakeBorder`)

Técnica	Archivo de la cátedra
Ecualizar un recorte con slicing, `pathlib.Path`, validar la carga, matriz booleana por umbral y `np.sum` por filas y columnas contra el máximo	`PDI_U2_p2_stain.py`
Umbral manual, `connectedComponentsWithStats` y `stats`, `sorted` con `key`, `findContours` con `RETR_TREE` y jerarquía, `cvtColor` a color, `cv2.rectangle`	`PDI_U3_Segmentacion.py`
Inicio y fin de pulsos con `np.diff` y `np.argwhere`, agrupar con `reshape`, recortes por índices, separar letras por columnas vacías, un archivo por elemento con f-string	`PDI_U1_p1_Letras.py`
`cv2.putText`	`PI_U2_ej_video_face_detection_and_blur.py`
`np.full` y el orden BGR de OpenCV	`PI_U2_ej_blurred_face.py`
`cv2.imwrite`	`PDI_U1_Fundamentos_p1.py`

Sin fuente en el material, por lo tanto decisiones o recursos propios que conviene poder justificar: los umbrales del Problema 2, el criterio de forma para L, R y A, las posiciones de los cuadrados medidas a mano, la escritura del CSV con `open` y `write`, y `np.vstack` para apilar las filas de la imagen de salida.

Ejemplo Práctico / Analogía

El concepto más difícil es por qué la ecualización local muestra lo que la global esconde. Pensalo como la nota por percentil en un curso. Ecualizar es reemplazar cada valor por su posición relativa dentro de un grupo: qué porcentaje del grupo tiene lo mismo o menos que vos.

- Ecualización global: tu percentil se calcula contra toda la facultad. Un 7 es un número común y queda en el medio de la tabla.
- Ecualización local: tu percentil se calcula solo contra tus compañeros de alrededor. Si en tu comisión todos sacaron entre 6 y 7, ese 7 te deja entre los primeros.

En la imagen pasa lo mismo. Un píxel de nivel 10 dentro de un cuadrado oscuro es uno entre millones de píxeles oscuros para el histograma global, y casi no se distingue. Dentro de una ventana de 21x21 sus vecinos van de 0 a 11 y él queda entre los más claros, así que sale con un valor alto (247 en el ejemplo de la ventana en (212, 207)).

La analogía también explica el ruido. Si en un grupo todos sacaron entre 6,9 y 7,1, el percentil igual los reparte de 0 a 100 por diferencias que no significan nada. Con una ventana chica pasa eso con el fondo claro y aparecen las manchas. Con una ventana enorme el grupo es casi toda la imagen y se vuelve a la ecualización global.

Dónde falla la analogía: los alumnos tienen un grupo fijo, mientras que cada píxel tiene su propia ventana, que se superpone con la del vecino y se recalcula en cada paso. Además, el valor de salida se reescala al rango de 0 a 255 y no se informa como porcentaje.

Autoevaluación Rápida

1. ¿Por qué la ecualización global no deja ver los detalles de la imagen del Problema 1, y qué cambia cuando el histograma se calcula dentro de una ventana?
2. ¿Qué pasa con el fondo claro y con los detalles si la ventana es de 3x3? ¿Y si es de 101x101? Explicá por qué ocurre en cada caso.

3. ¿Por qué los umbrales para encontrar las líneas de la tabla se definen como fracción del máximo y no como un valor fijo de píxeles? ¿Por qué el de columnas es más bajo que el de filas?
4. Una celda de Nombre y Apellido tiene 12 componentes conectadas y la mayor separación entre dos consecutivas es de 3 píxeles. ¿Cuántas palabras cuenta el script y cómo queda el campo?
5. ¿Cómo distingue el script una L, una R y una A sin reconocer texto? ¿Qué medida usa para separar la L de la R, que empiezan igual?